

Guide CodeMachine

Version 26.2.0 10 mars 2026

Geneviève Cyr GIGL | Polytechnique Montréal

Ce document est distribué sous licence Creative Commons Attribution 4.0 International (CC BY 4.0). Vous êtes libre de le partager, copier, distribuer et adapter, y compris à des fins commerciales, à condition d'attribuer correctement la paternité en citant les auteurs originaux. N.B. Le masculin est utilisé pour alléger le texte.

Installation

Installation de Java

Avant de vous lancer à installer CodeMachine, vous devez vous assurer d'avoir installé **Java Runtime 8 (JRE 8)**.

🔗 Le JRE est suffisant puisqu'il permet d'**exécuter** des applications Java sans avoir besoin des outils de développement.

Vous pouvez télécharger la version la plus récente du **Java Runtime** (toutes plateformes) sur le site officiel :

🔗 [Télécharger Java Runtime 8](#)

⚠ Si vous tenez à installer le **JDK complet** (qui inclut le compilateur et l'environnement d'exécution), assurez-vous de choisir une **version 17 ou inférieure** (Windows, Linux ou macOS).

Le JDK est nécessaire uniquement si vous comptez développer en Java, et non simplement exécuter CodeMachine.

Pour Windows

1. Aller sur GitHub : [Page GitHub de CodeMachine](#)
2. Choisir le relâche la plus récente (cliquer dessus)

The screenshot shows the GitHub repository page for 'code-machine-v2'. The repository is public and has 4 branches and 2 tags. The 'Releases' section on the right highlights the latest release 'v25.1.0: Dev (#25)' with a 'Latest' badge. The 'README' section on the left describes the application as a simple processor simulation used in the INF1600 course at Polytechnique Montréal. The 'Frontend' section mentions the use of Electron and a regular browser.

3. Choisir la bonne architecture (celle correspondant à votre ordinateur) et cliquer dessus pour la télécharger

v25.1.0: Dev (#25) Latest Compare

github-actions released this 1 hour ago · 1 commit to main since this release · v25.1.0 · 6a5f525

CodeMachine v25.1.0

Première version de Code Machine en nouvelle version et standalone, contient la compilation et une version basique de l'écriture de code

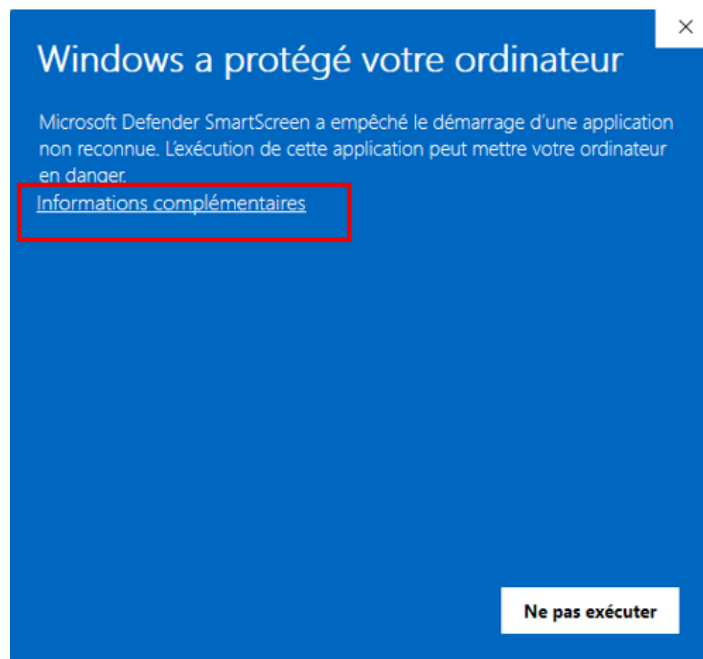
▼ Assets 7

CodeMachine-v25.1.0-linux-arm64.AppImage	sha256:49d299f09f618e12c38e9fa64171d...	161 MB	2 hours ago
CodeMachine-v25.1.0-linux-x86_64.AppImage	sha256:810928767e5298e29b13ed51cd786...	160 MB	2 hours ago
CodeMachine-v25.1.0-mac-universal.dmg	sha256:92fb66c5ec463397b49154d2d533c...	239 MB	2 hours ago
CodeMachine-v25.1.0-win-arm64.msi	sha256:8fa2e03d1ab59441161badb63a31a...	139 MB	1 hour ago
CodeMachine-v25.1.0-win-x64.msi	sha256:830735a6f506a4e6c4ba804f9dda4...	143 MB	1 hour ago
Source code (zip)			2 hours ago
Source code (tar.gz)			2 hours ago

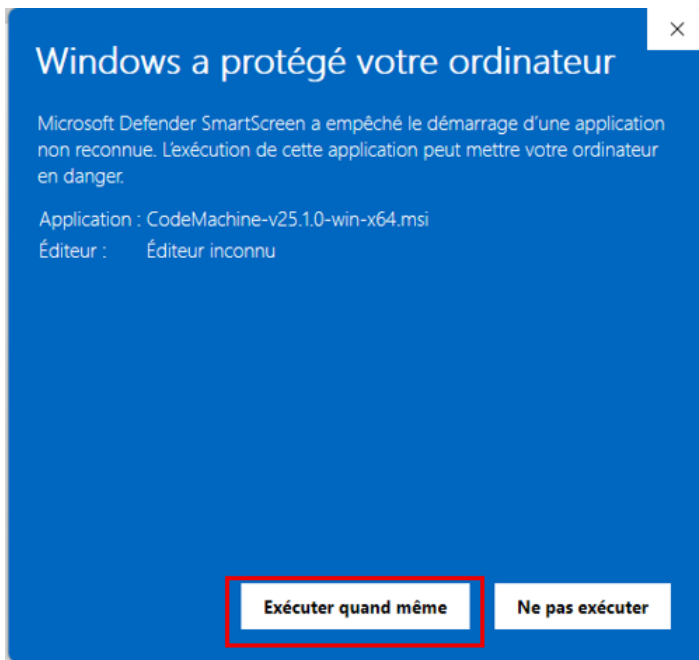
un des deux

4. Dans téléchargement, double-cliquer pour partir l'installation.

5. Cliquer sur Informations complémentaires

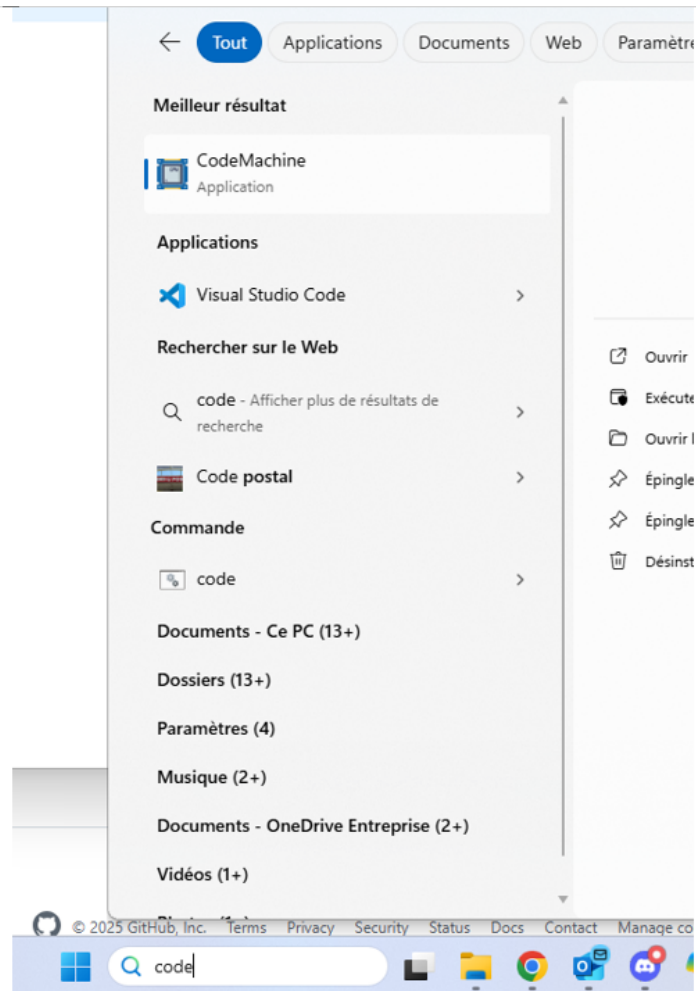


6. Choisir exécuter quand même



7. Suivre les étapes d'installations

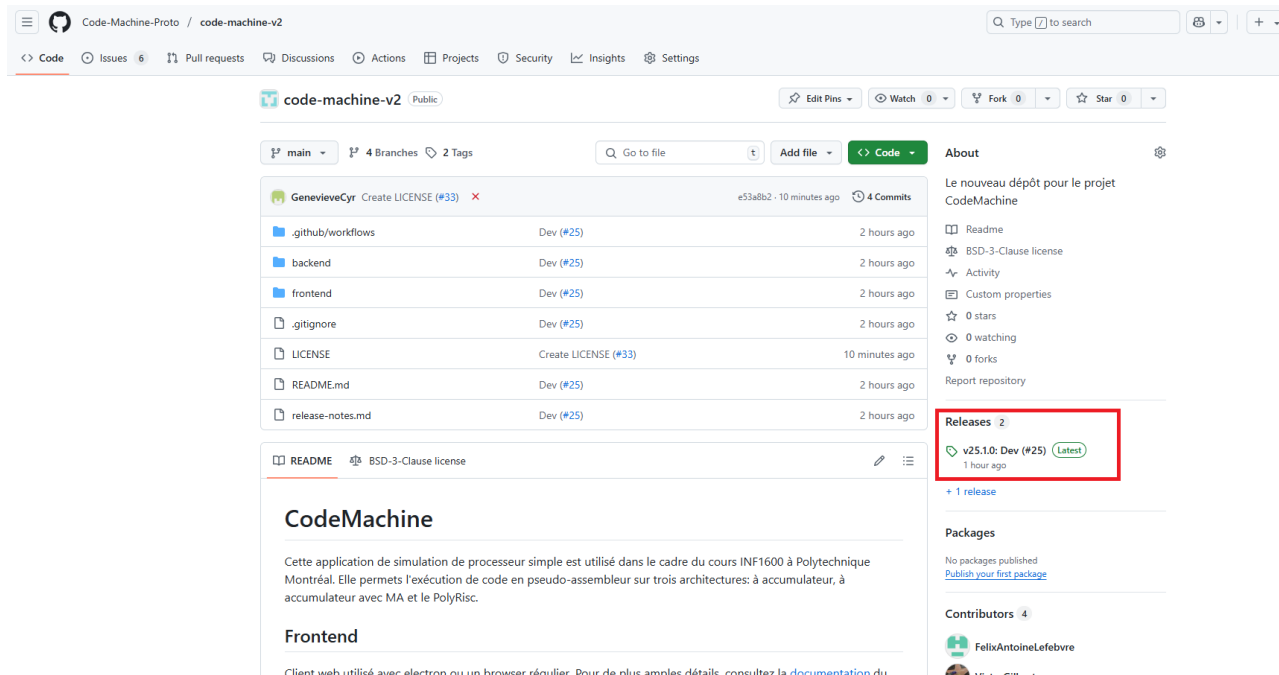
8. Vous trouverez CodeMachine dans la barre de recherche



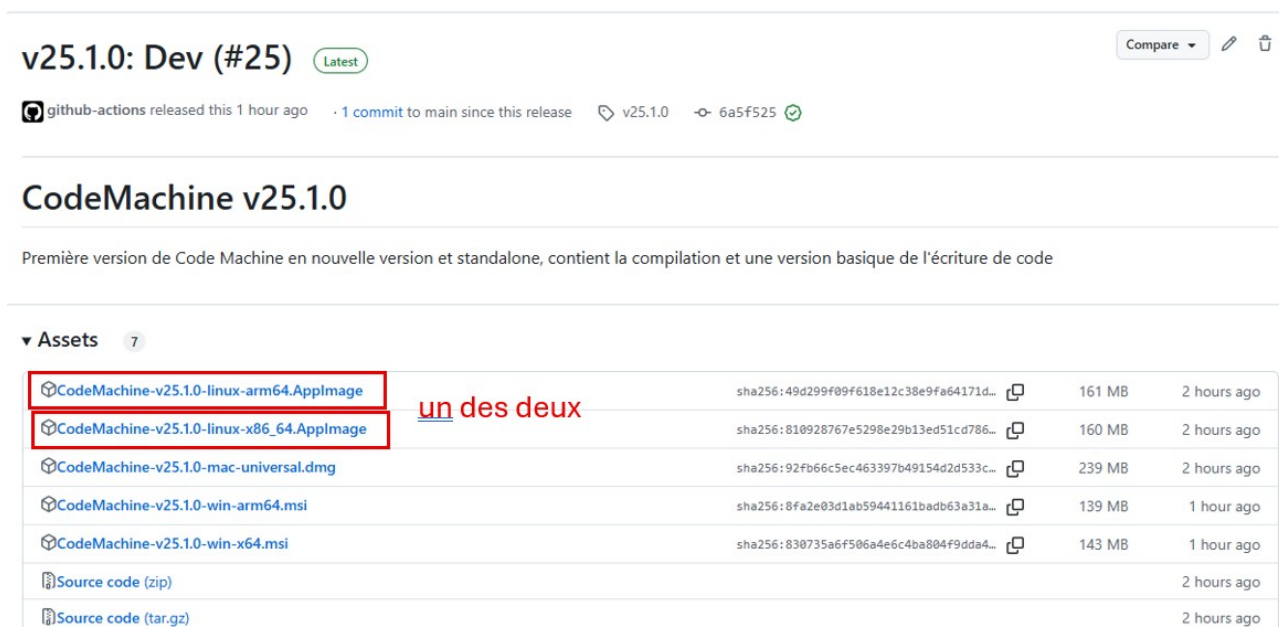
Pour Linux

1. Aller sur GitHub : [Page GitHub de CodeMachine](#)

2. Choisir le relâche la plus récente (cliquer dessus)

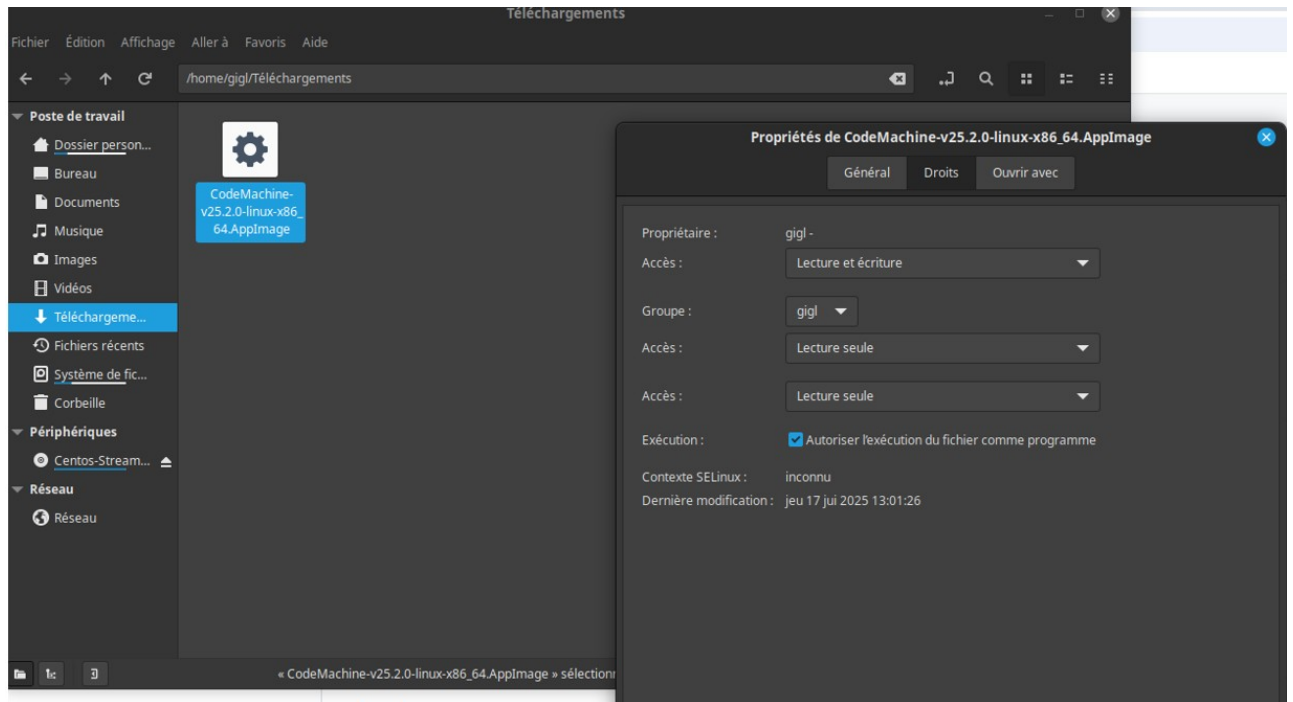


3. Choisir la bonne architecture (celle correspondant à votre ordinateur) et cliquer dessus pour la télécharger



4. Dans téléchargement, double-cliquer pour partir l'installation.

5. Ensuite, vous avez deux options, soit aller dans téléchargement et s'assurer que le fichier est exécutable, puis **double-cliquer pour partir CodeMachine (aucune installation requise)**.



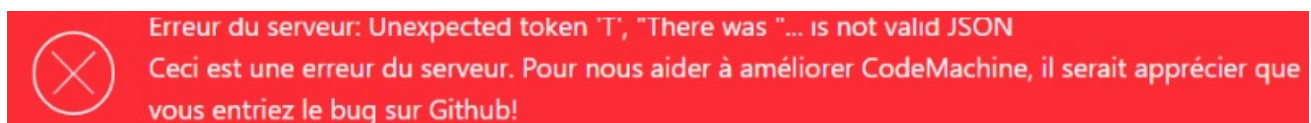
Sinon, vous pouvez aller **par le CLI**, changer les permissions et partir l'outil comme suit :

```
[gigl@localhost Téléchargements]$ chmod 777 CodeMachine-v25.2.0-linux-x86_64.AppImage
[gigl@localhost Téléchargements]$ ./CodeMachine-v25.2.0-linux-x86_64.AppImage &
[1] 60577
```

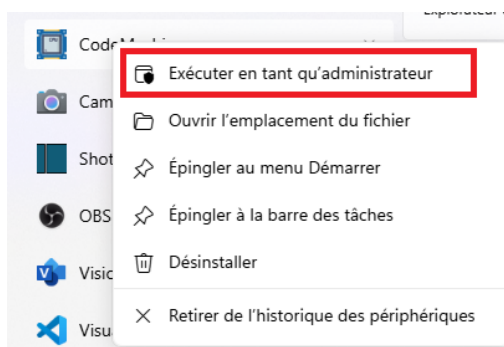
Notez que vous pouvez déplacer le .AppImage à l'endroit que vous préférez pour faciliter son accès. Cependant, vous devrez, dans tous les cas, permettre son exécution en changeant ses permissions.

ATTENTION : Sur Windows, dans certaines installations, il faut parfois partir CodeMachine en mode administrateur pour qu'il compile correctement

Si vous avez l'erreur suivante au moment de la compilation :

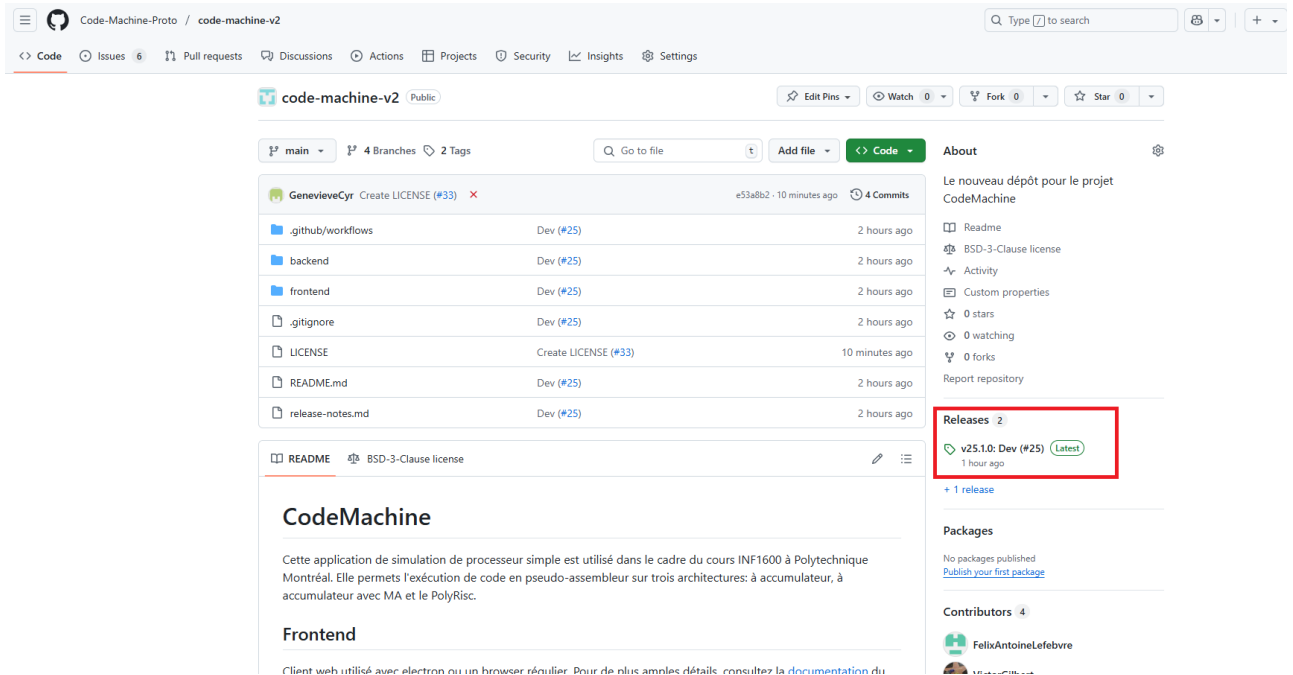


Il faut repartir CodeMachine en tant qu'administrateur en utilisant le bouton contextuelle de la souris.

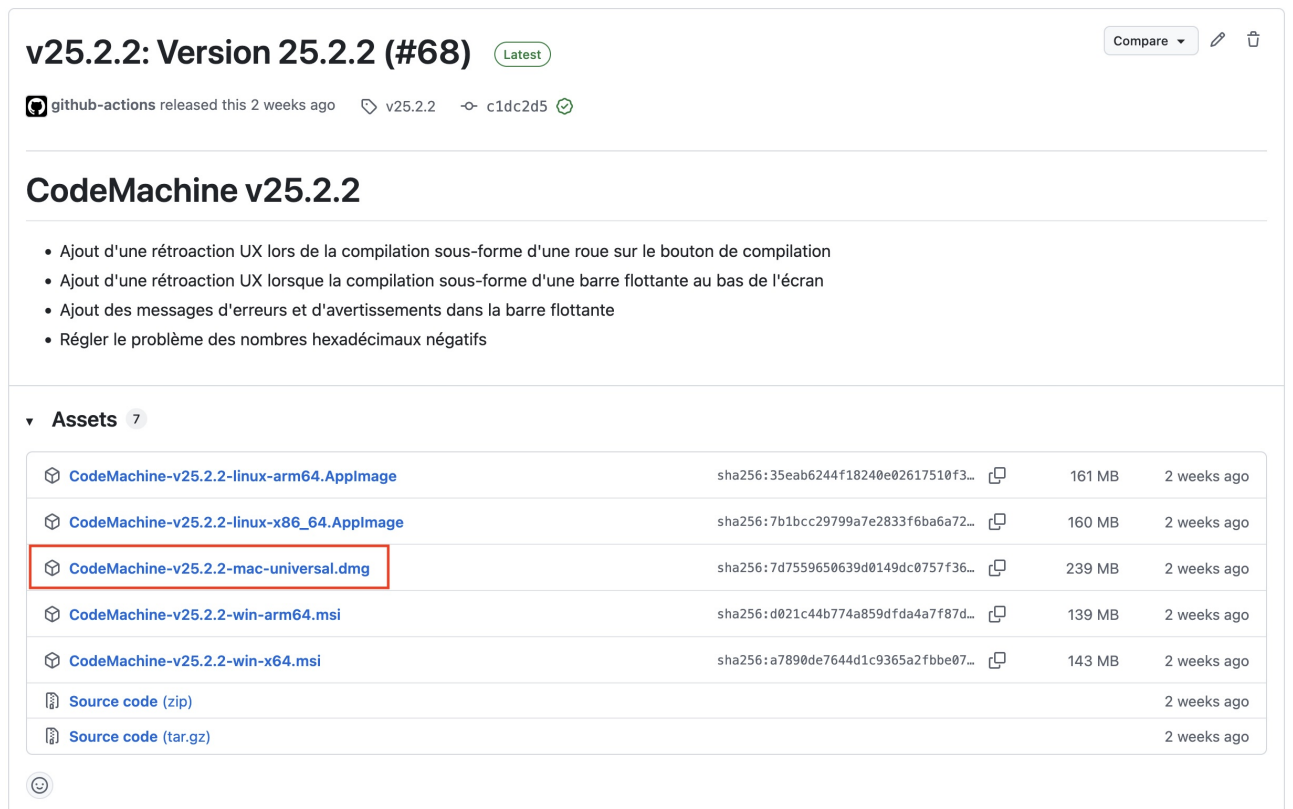


Pour MAC

1. Aller sur GitHub : [Page GitHub de CodeMachine](#)
2. Choisir le relâche la plus récente (cliquer dessus)



3. Choisir l'installateur pour Mac (le même pour toutes les architectures)

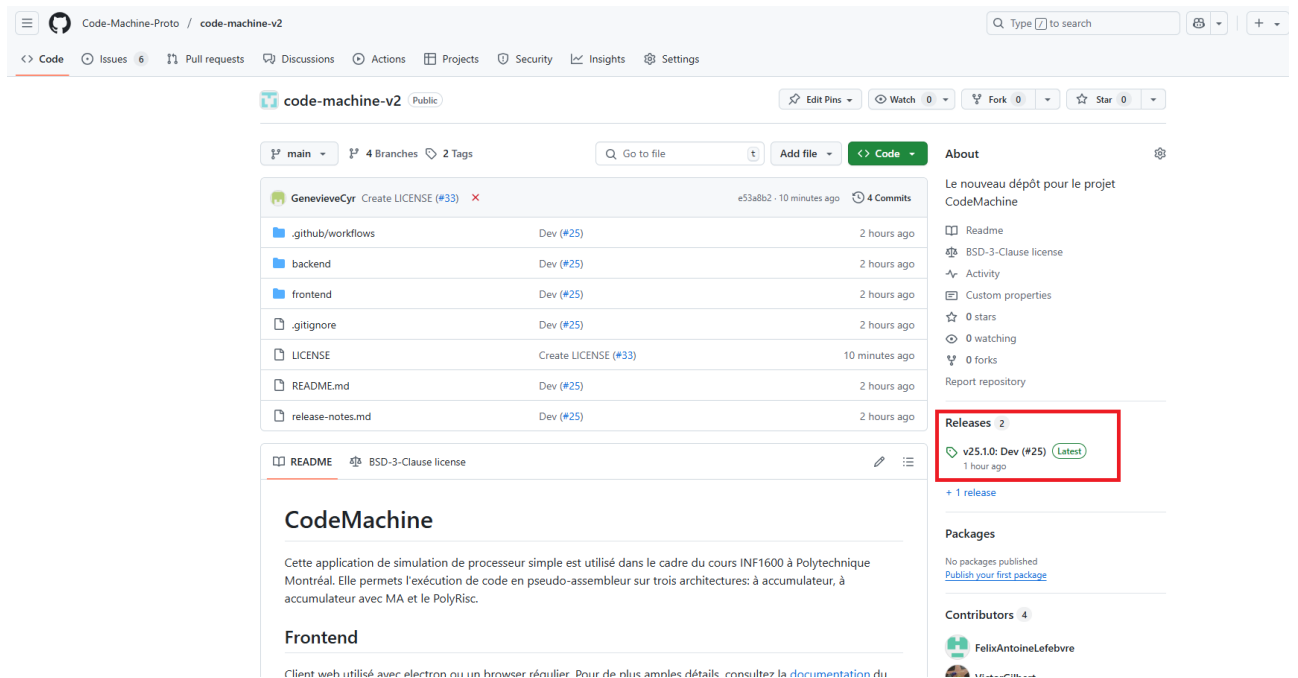


4. Dans téléchargement, double-cliquer pour partir l'installation et compléter une installation selon le format dmg

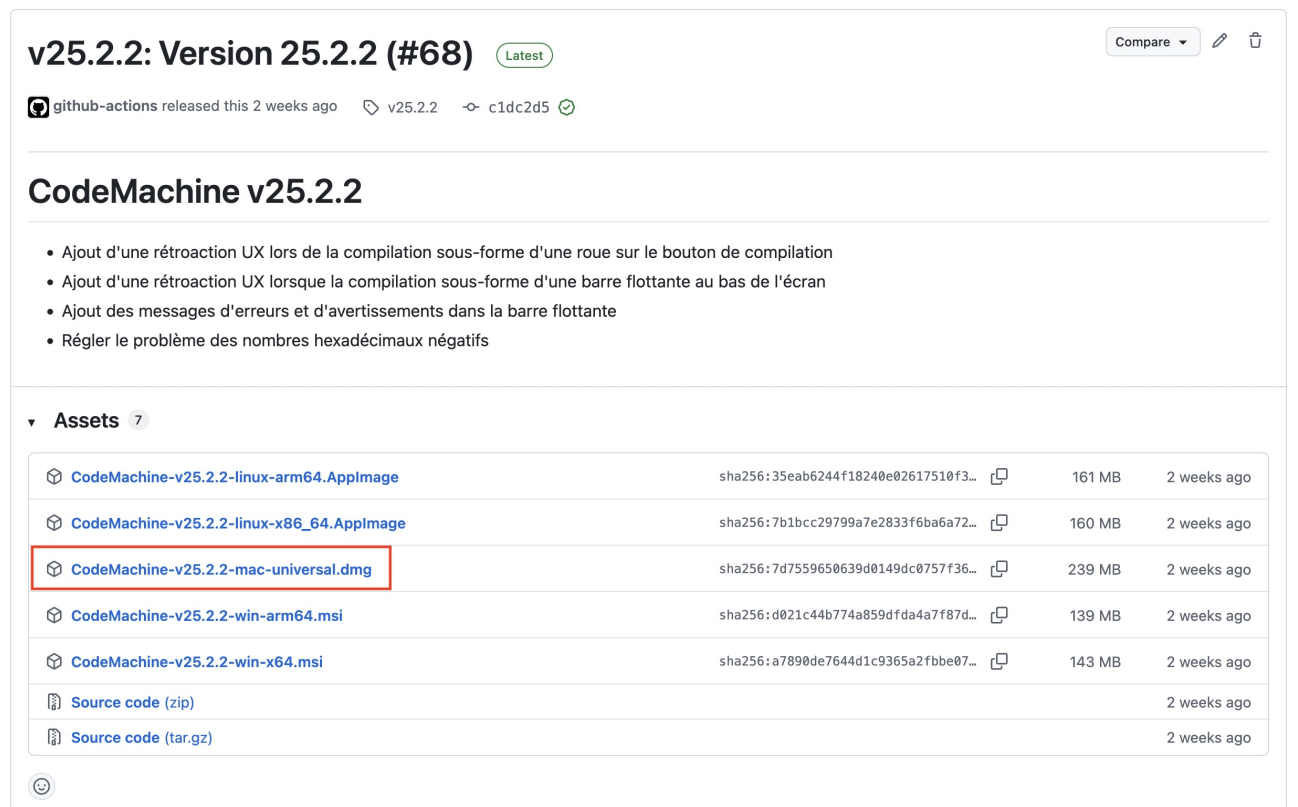
Méthode 2: Installation via Terminal (méthode alternative)

1. Aller sur GitHub : [Page GitHub de CodeMachine](#)

2. Choisir le relâche la plus récente (cliquer dessus)



3. Choisir l'installateur pour Mac (le même pour toutes les architectures)

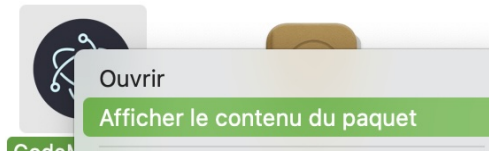


4. Dans téléchargement, double-cliquer pour partir l'installation et compléter une installation selon le format dmg

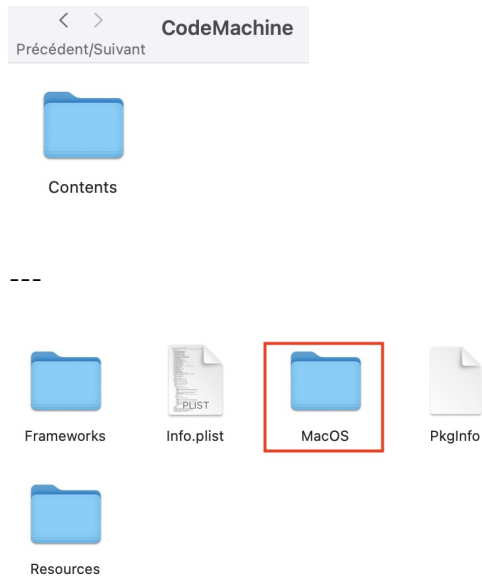
5. Enlever les drapeaux de quarantaine mis par Apple

```
felix-antoinelefebvre@MacBook-Air-de-Felix-Antoine-427 ~ % cd /Applications
felix-antoinelefebvre@MacBook-Air-de-Felix-Antoine-427 /Applications % xattr -r -d com.apple.quarantine CodeMachine.app
felix-antoinelefebvre@MacBook-Air-de-Felix-Antoine-427 /Applications %
```

6. Trouver l'application dans le Finder et afficher le contenu du paquet



7. Naviguer Contents > MacOS



8. Double-cliquer sur l'exécutable nommé CodeMachine pour le partir en mode administrateur, créer un alias pour mettre sur votre bureau est fortement recommandé



Guide d'utilisation de CodeMachine

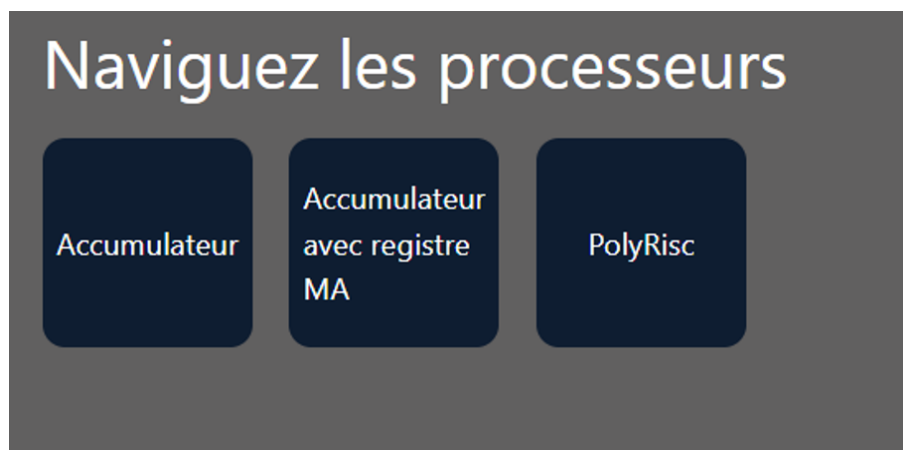
Utilisation de l'interface graphique

Version

Vous pourrez toujours savoir quelle version vous utilisez en regardant la version à cause de « Code Machine ». Normalement, l'outil fonctionnel devrait avoir une version supérieure ou égale à 25.2.2. Cependant, vous devriez toujours prendre la dernière version sur GitHub et vous pouvez voir les commentaires des modifications faites sur les versions directement dans Git.

Architectures

Trois architectures sont disponibles dans CodeMachine : Accumulateur, Accumulateur-MA et PolyRisc. Vous pouvez choisir l'architecture en cliquant sur le gros bouton.



Raccourcis

- Pour faire un "Zoom In" : ctrl + (souvent ctrl-shift=)
- Pour faire un "Zoom out" : ctrl –
- Vous pouvez être en mode « plein écran » ou non à votre choix.
- Lorsque vous écrivez le code, vous pouvez utiliser « ctrl-Z » et « ctrl-y » pour annuler ou répéter une frappe.
- Pour revenir au menu principal (donc quitter une architecture pour aller dans un autre), appuyer sur la flèche blanche ou directement sur Code Machine.



Compilation

- Le dernier code entré dans chaque architecture devrait être mémorisé quand vous quittez l'architecture. Cependant, dès que vous quittez une architecture, en retournant dans n'importe quelle architecture, il faut recompiler. Pour savoir si le code est compilé, regardez le / en haut.

0 /0

Indique un code non-compilé (donc un /0)

0 /219

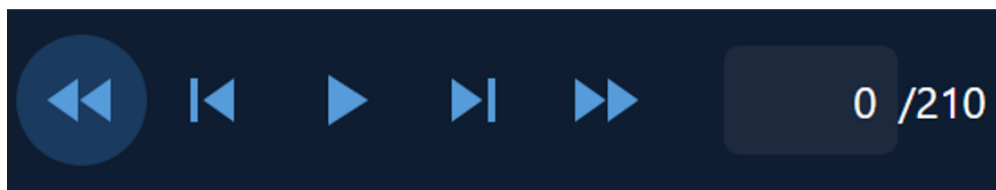
Indique un code compilé (donc un / > 0)

- Durant la compilation, le bouton « Compiler » aura un indicateur de chargement et le bouton sera inaccessible. Le temps de compilation pour les architectures « Accumulateur » sont relativement rapide. Pour le « PolyRisc », c'est plus long (cela peut prendre au-delà d'une minute pour plusieurs cycles)

Affichage du nombre de cycles

ATTENTION : CodeMachine est limité à 1024 cycles au total. Tout code qui donnera plus de 1024 cycles, seulement les 1024 premiers cycles seront exécutés.

- Pour l'Accumulateur et l'Accumulateur-MA, les cycles sont comptés à partir de 0, donc le nombre total de cycle (/nb) sera toujours égale au nombre de cycle -1. Pour le PolyRisc, les cycles sont comptés à partir de 1, donc le nombre total de cycle (/nb) sera toujours égale au nombre de cycle -1.
- Pour toutes les architectures, chaque instruction prend 3 cycles (incluant l'instruction « nop »).
- Pour naviguer dans le code vous avez plusieurs options.
 - Utiliser le « play » (triangle bleu simple) et le code sera exécuté automatiquement, étapes par étapes.
 - Utiliser les boutons « next step » ou « previous step » (triangle bleu avec barre verticale), pour exécuter le code une étape à la fois.
 - Utiliser les boutons « goto end » ou « goto start » (double triangle bleu), pour aller directement à la fin ou au début du code.
 - Décider exactement à quel cycle aller en entrant le nombre de cycle à la place du chiffre qui apparaît avant le « / »



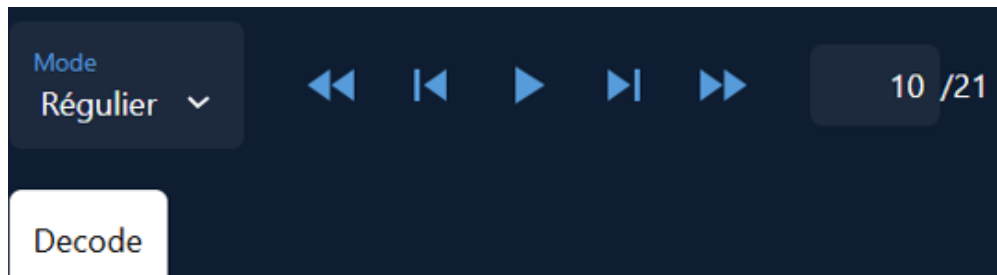
- Lorsque vous utilisez le mode « régulier », une étape correspond à un cycle. Donc chaque instruction passera par les étapes : « fetch », « decode », « execute » (3 cycles).

Mode
Régulier ▾

- Lorsque vous utilisez le mode « exécution », chaque étape correspond à une instruction. Ainsi, chaque étape passera d'un cycle « execute » d'une instruction à l'autre (par exemple de 3 cycles).

Mode
Exécution ▾

- Le cycle auquel vous êtes rendu est toujours affiché dans le nombre avant le « / ». Attention de penser à additionner un à ce nombre pour savoir exactement à quel cycle vous êtes rendu (puisque les cycles sont comptés à partir de 0). Vous pouvez aussi, en tout temps, voir à quel stade d'exécution de l'instruction vous êtes dans le petit rectangle blanc : « fetch », « decode », « execute ».



Surbrillance du code et erreur de syntaxes

- Normalement, les instructions sont en *rouge*, les « déclarations » d'étiquettes en *mauve* et les registres en *orange* et les valeurs ou « utilisation » d'étiquettes sont en *blanc* dans votre code.

```
1  .text
2  ldi r1, 4
3  ldi r2, 2
4  ldi r4, 8
5  loop:
6  sub r1, r1, r2
7  brnz loop
8  st (r4), r1
9  stop
10
```

- Les erreurs de syntaxes devraient être soulignées en *rouge* et les "warnings" en *jaune* et tant que vous avez des soulignements rouges dans le code, vous ne pourrez pas accéder au bouton « Compiler » qui

sera *rouge*. Normalement, lorsque vous avez des soulignées dans le code, une fenêtre devrait apparaître et vous indique le type d'erreur précédé du numéro de la ligne qui contient une erreur.

ATTENTION : Lorsqu'il y a une erreur dans une ligne de code, il se pourrait que la surbrillance des erreurs des lignes suivantes ne soient pas exactes. Il est très important de régler les premières erreurs dans le code pour pouvoir continuer la correction des lignes suivantes

- Si vous compilez et qu'une erreur se produit (qui n'a pas été détectée par le « parser »), un message vous l'indiquera, mais vous devrez trouver sans aide le problème de votre côté.

Ne vous gênez pas pour ouvrir des "issues" sur GitHub si ce genre de situation se produit pour qu'on puisse améliorer l'outil.

- Les règles d'écriture du code sont données dans la section « Grammaire du code ».
- Si votre code est trop long pour s'afficher au complet à l'écran, vous devez utiliser la roulette de la souris pour faire défiler le code. Il n'y a pas de barre de défilement.
- Vous pouvez voir l'instruction en cours d'exécution par sa surbrillance.

Organisation mémoire

- **Les numéros de ligne** dans votre code assembleur **ne correspondent PAS** aux adresses mémoires réelles
- **Les directives** (`.text`, `.data`) ne sont **pas écrites en mémoire** - elles indiquent seulement au compilateur comment organiser les sections
- **Ordre en mémoire:**
 1. Section `.text` (programme) → placée en premier en mémoire
 2. Section `.data` (données) → placée après le code
- **Les étiquettes** (comme `loop:`) ne sont **pas écrites en mémoire** - elles sont remplacées par l'adresse de l'instruction suivante lors de l'assemblage

Exemple:

```
.text                # Cette directive n'occupe pas de mémoire
ld n                # Occupe de la mémoire (adresse 0)
loop:               # Remplacée par l'adresse réelle (adresse 1)
sub one             # Occupe de la mémoire (adresse 1)
brnz loop           # Occupe de la mémoire (adresse 2), "loop" → adresse 1
st n                # Occupe de la mémoire (adresse 3)
stop                # Occupe de la mémoire (adresse 4)

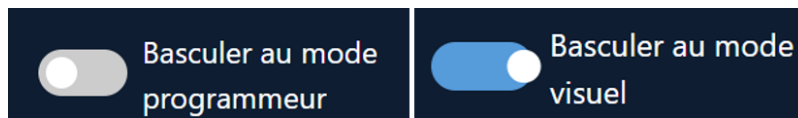
.data               # Cette directive n'occupe pas de mémoire
n: 5                # Occupe de la mémoire (adresse 5)
one: 1              # Occupe de la mémoire (adresse 6)
```

Dans cet exemple:

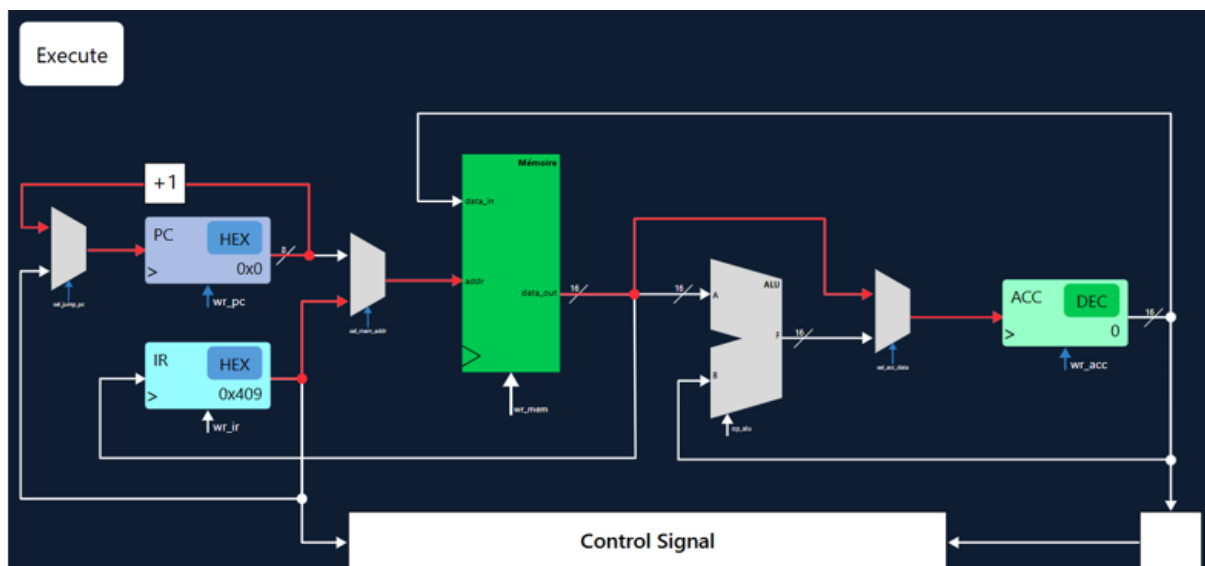
- Les instructions (**.text**) occupent les adresses 0 à 4
- Les données (**.data**) occupent les adresses 5 et 6
- L'étiquette **loop:** est remplacée par l'adresse 1
- Les directives **.text** et **.data** n'occupent aucun espace mémoire

Mode d'affichage graphique

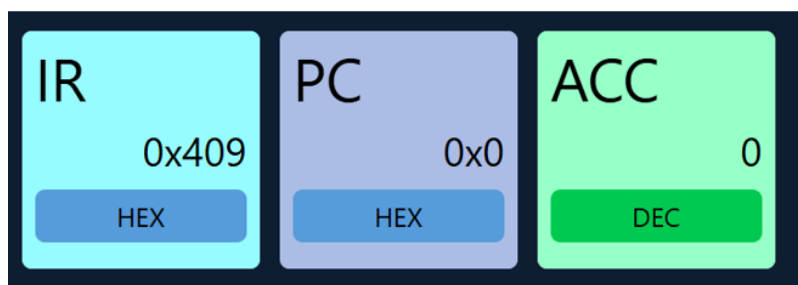
- Chaque architecture à 2 modes d'affichage graphique : mode « visuel » et mode « programmeur ».
Contrôlable avec le bouton suivant :



- En mode « visuel », vous verrez le circuit matériel avec les différents blocs et les signaux qui sont actifs au cycle en cours seront en rouge.

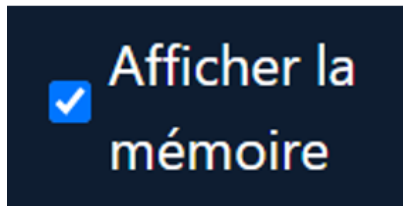


- En mode « programmeur », vous verrez le contenu des registres, sans voir le circuit :



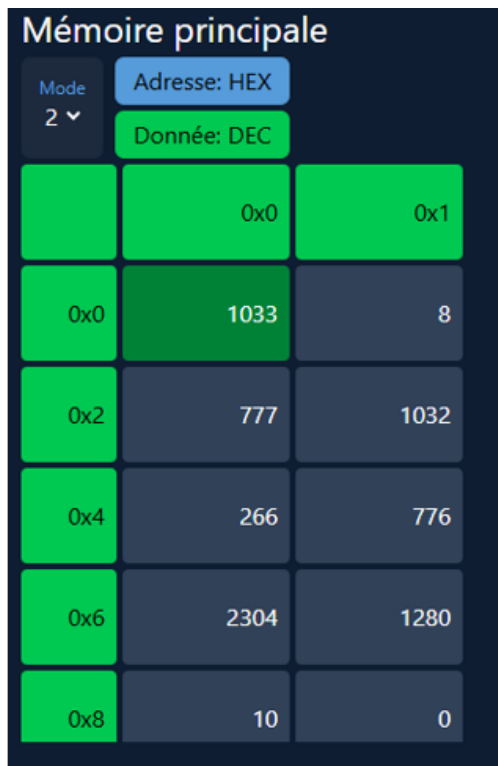
Affichage de la mémoire

- Il est possible d'afficher ou masquer le contenu de la mémoire principale avec le bouton :



Ce bouton est disponible en mode « visuel » ou en mode « programmeur » mais l'espace occupé par chaque partie sera adapté à l'espace d'écran disponible. À vous de trouver ce qui vous convient le mieux.

- Le contenu de la mémoire ne s'affichera que lorsqu'un programme est compilé.
- Pour les deux premières architectures, il n'y a qu'une mémoire qui contient les données et les instructions.

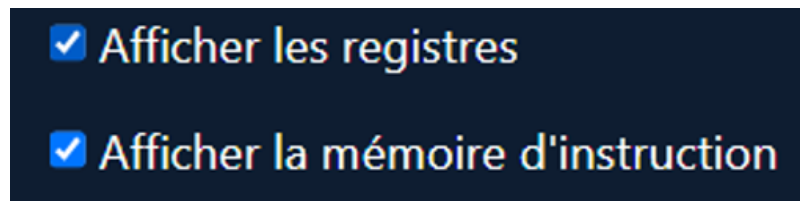
L'interface 'Mémoire principale' est présentée sur un fond noir. En haut à gauche, un menu déroulant 'Mode' est réglé sur '2'. À sa droite, deux boutons sont présents : 'Adresse: HEX' (bleu) et 'Donnée: DEC' (vert). En dessous, une table à 3 colonnes et 6 lignes est visible. Les cellules de la première colonne ont un fond vert, celles de la deuxième un fond vert foncé, et celles de la troisième un fond gris foncé.

	0x0	0x1
0x0	1033	8
0x2	777	1032
0x4	266	776
0x6	2304	1280
0x8	10	0

- Il est possible de choisir si les adresses et les données sont en décimales ou en hexadécimales avec les deux boutons dans le haut de la mémoire.
- Vous pouvez aussi choisir le « mode » d'affichage (1, 2 ou 4) qui vous mettra 1, 2 ou 4 adresses par ligne (selon votre préférence).
- Vous remarquerez qu'il y a des adresses pour chaque ligne et chaque colonne. En fait l'adresse mémoire d'une valeur (exemple : 776) correspond à la somme de l'adresse de la ligne et de la colonne (exemple : $0x4 + 0x1 = 0x5$ pour la donnée 776). Attention, dans CodeMachine les données sont TOUJOURS de 16 bits (2 octets) et ne sont pas accessibles par octet. Chaque adresse mémoire pointe

sur une case mémoire d'une grandeur de 16 bits. Ce sont ces cases que vous voyez dans chaque rectangle.

- Pour se promener dans la mémoire et faire défiler les adresses, il n'y a pas de barre de défilement, vous devez utiliser la roulette de la souris pour faire défiler la mémoire.
- Pour le Polydisc, l'affichage de la mémoire principale de données fonctionne de la même manière que les deux autres architectures. Cependant, le Polydisc a aussi une « mémoire » de registres à usage général ainsi qu'une mémoire programme. Ces deux mémoires supplémentaires ne sont accessibles qu'en mode « programmeur » à l'aide de deux cases à cocher selon ce que vous voulez voir.



- Le fonctionnement de l'affichage des 2 autres mémoires est exactement le même que pour la mémoire principale. Cependant, notez que la largeur des données de la mémoire d'instruction du Polydisc est de 28 bits et non de 16 bits comme toutes les autres mémoires.
- Vous pouvez donc, en mode « programmeur » voir le contenu des 3 mémoires comme suit :

Registres			Mémoire d'instruction			Mémoire principale		
Mode 2 ▾	Adresse: HEX	Donnée: DEC	Mode 2 ▾	Adresse: HEX	Donnée: HEX	Mode 2 ▾	Adresse: HEX	Donnée: DEC
	0x0	0x1		0x0	0x1		0x0	0x1
0x0	0	0	0x0	0xa010004	0xa020002	0x0	0	0
0x2	0	0	0x2	0xa040008	0x1010102	0x2	0	0
0x4	0	0	0x4	0xc020003	0x9000401	0x4	0	0
0x6	0	0	0x6	0xf000000	0x0	0x6	0	0
0x8	0	0	0x8	0x0	0x0	0x8	0	0

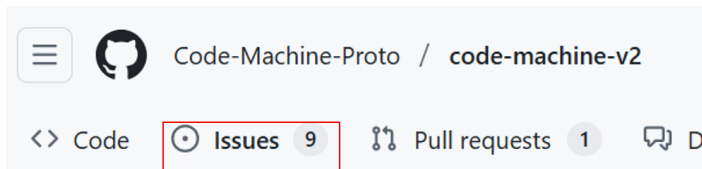
Grammaire du code

- Le code devra toujours suivre les règles suivantes :
- La directive « .data » est facultative et une directive « .data » vide est acceptée.
- Les étiquettes (« label ») doivent être constituées de seulement des lettres minuscules, des lettres majuscules, des chiffres et des tirets du bas(_) (attention, les étiquettes sont sensibles à la case)
- Les instructions doivent être en minuscules seulement (tout est sensible à la case)
- La déclaration des étiquettes doit toujours mettre le « : » sans espace entre l'étiquette et le « : » (Ex : loop: et non loop :)
- Les nombres entrés dans le code, comme valeur, doivent toujours être des nombres entiers (négatif ou non)
- Chaque instruction doit être séparée par un « new line » (retour de charriot)
- A part pour la déclaration d'étiquettes, il n'y a aucune dépendance aux espaces
- Les lignes vides à la fin du code ne sont pas problématiques.
- Les commentaires peuvent utiliser le « # » ou les « // » et ce n'importe où dans la ligne (sans code ou après le code)

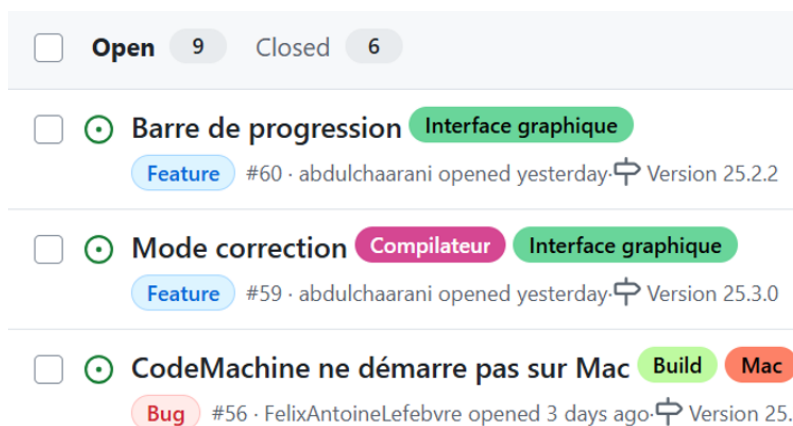
Comment entrer des « issues » sur CodeMachine.

Si vous avez des problèmes avec CodeMachine, vous pouvez les souligner aux développeurs. Cela se fera directement sur GitHub.

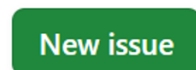
1. Vous devez d'abord aller dans « Issues » :



2. Regarder si votre problème n'a pas été déjà entrée en lisant les « open » issues



3. Aller dans « New Issue »



4. Choisir « bug » si c'est un problème avec ce qui est déjà implanté ou « nouvelle fonctionnalité » si vous avez une demande de modifications de CodeMachine (dans sa fonctionnalité).



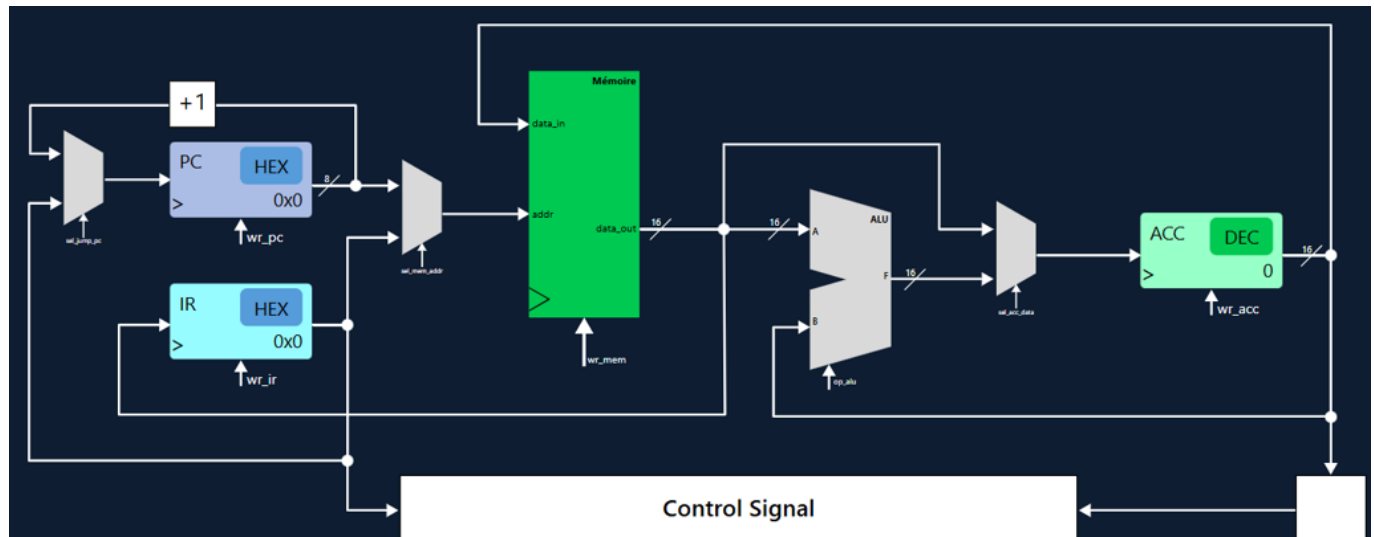
5. Dans chacun des cas, il y a un « template » de base qui vous guide dans l'information à entrer dans votre « issue » pour faciliter la compréhension du développeur. SVP, suivez ces instructions pour faciliter leur travail.

Notez que vous avez accès au code (c'est « open source »), alors vous pouvez faire un clone et jouer dans le code à votre aise si vous le désirez. Vous avez aussi une manière de déployer votre version automatiquement (pour vous-même), mais nous ne supporterons pas le code, juste l'interface. De plus, les accès en écriture sont proscrits, seulement les développeurs pourront changer le code en cours.

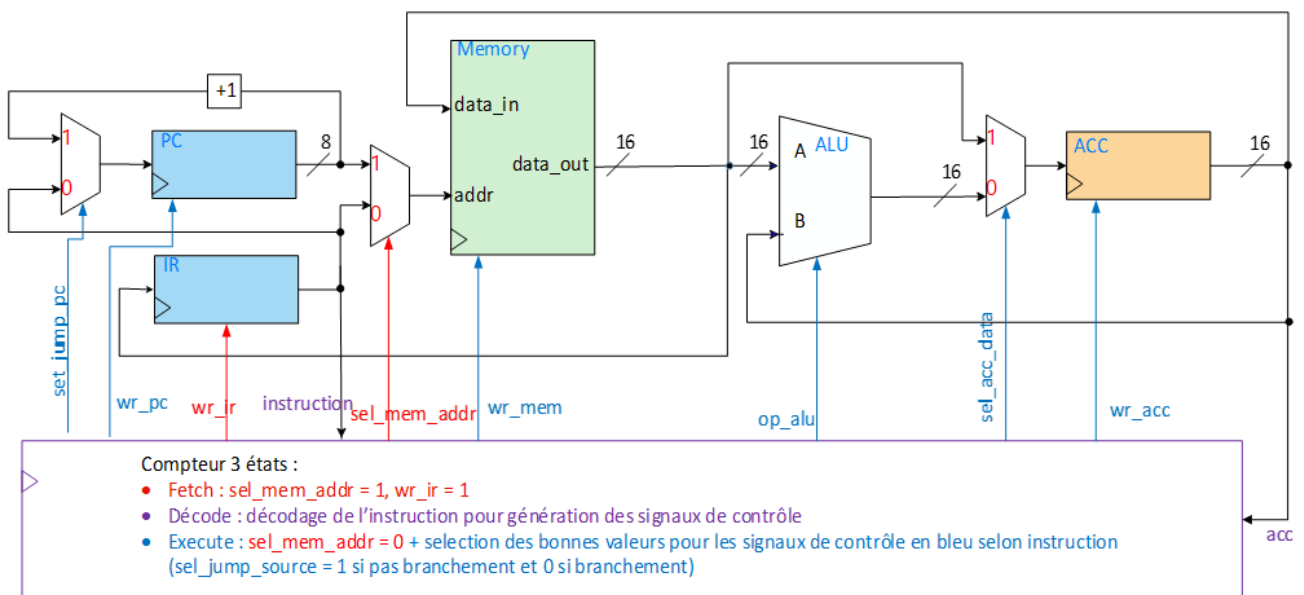
Architecture Processeur-accumulateur

Circuit logique

CodeMachine



Détaillé



Instructions

Instruction	Encodage	Description
add ADR	0x00XX	ACC <- ACC + Mémoire[ADR]
sub ADR	0x01XX	ACC <- ACC - Mémoire[ADR]
mul ADR	0x02XX	ACC <- ACC × Mémoire[ADR]
st ADR	0x03XX	Mémoire[ADR] <- ACC
ld ADR	0x04XX	ACC <- Mémoire[ADR]
stop	0x05XX	Arrêt du programme
br ADR	0x07XX	PC <- ADR
brz ADR	0x08XX	ACC = 0 ? PC <- ADR : PC <- PC + 1
brnz ADR	0x09XX	ACC != 0 ? PC <- ADR : PC <- PC + 1

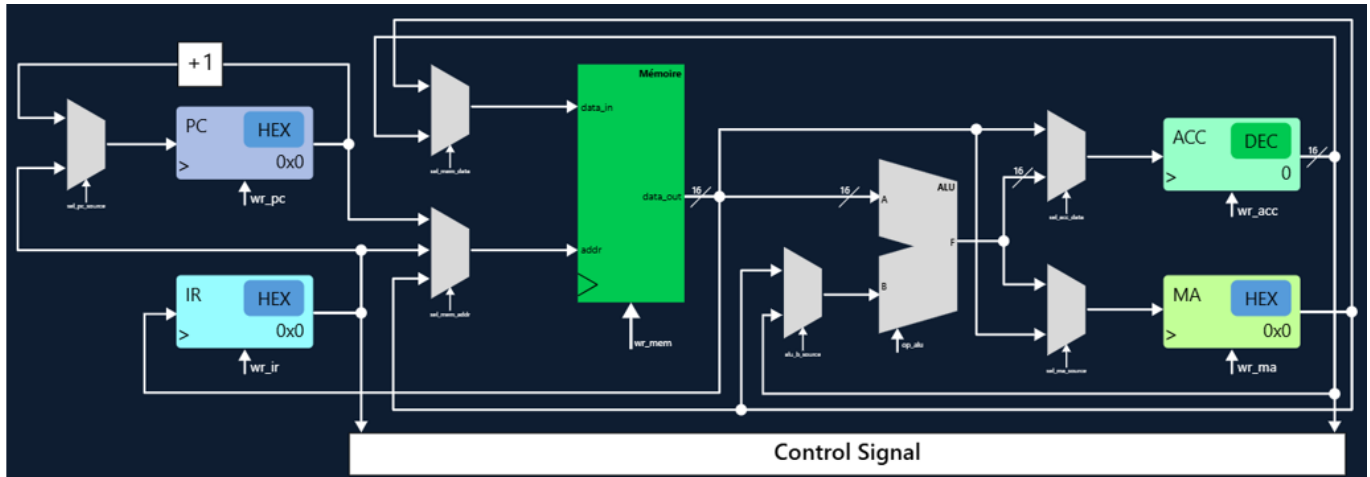
Opérations ALU

op_alu	opération
0	B + A
1	B – A
2	B x A

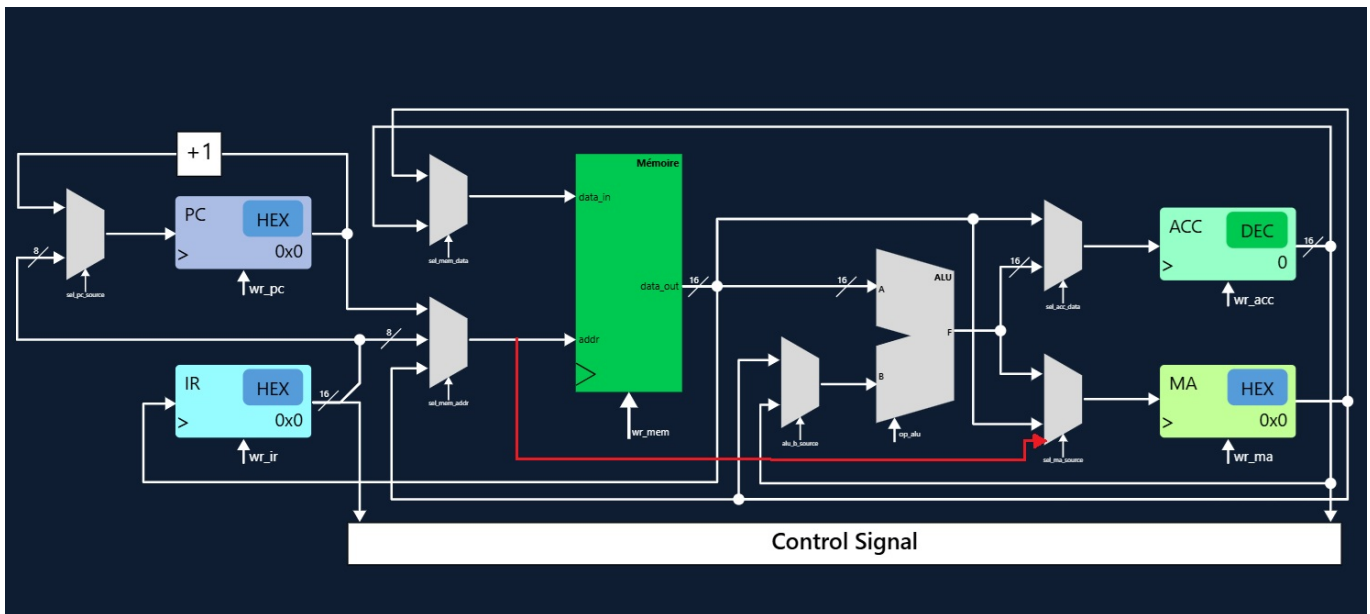
Architecture Processeur-accumulateur-MA

Circuit logique

CodeMachine

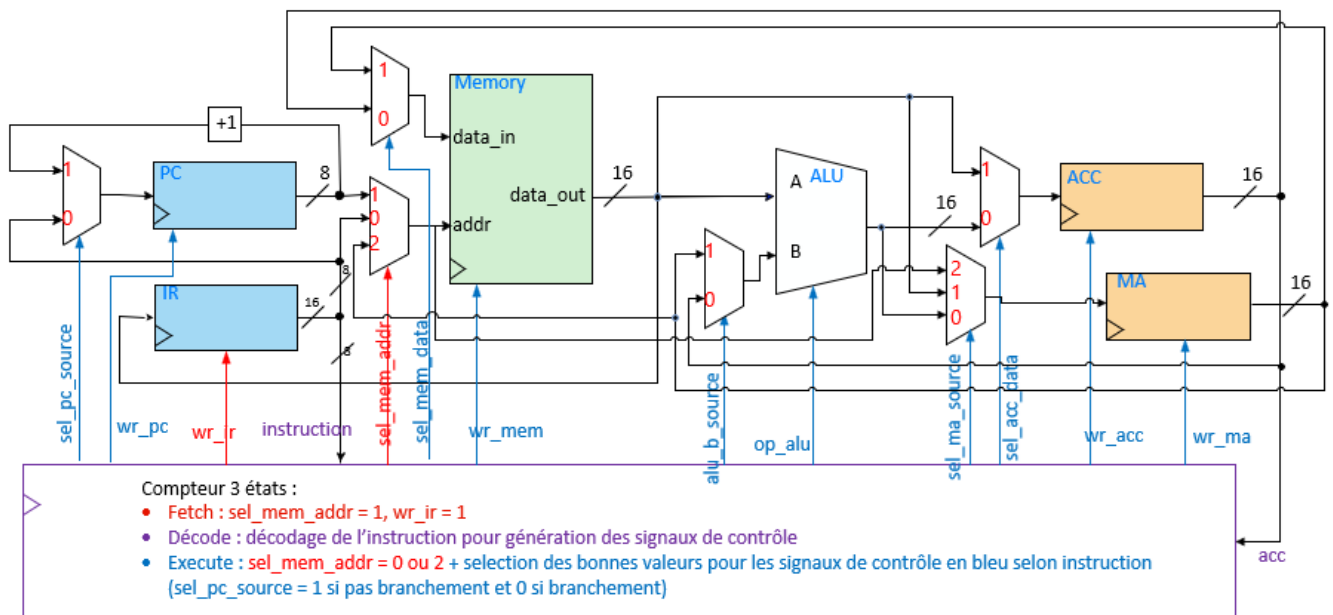


Attention: Une nouvelle instruction (lea) a été ajoutée dans le jeu d'instructions et n'est pas encore représentée dans le schéma de CodeMachine. Voici ce qu'il manque et sera ajouté éventuellement dans l'interface graphique.



Pour plus de détails, consultez [l'issue #123](#) sur GitHub.

Détailé



Instructions

Instruction	Encodage	Description
add ADR	0x00XX	ACC <- ACC + Mémoire[ADR]
sub ADR	0x01XX	ACC <- ACC - Mémoire[ADR]
mul ADR	0x02XX	ACC <- ACC × Mémoire[ADR]
adda ADR	0x03XX	MA <- MA + Mémoire[ADR]
suba ADR	0x04XX	MA <- MA - Mémoire[ADR]
addx	0x05XX	ACC <- ACC + Mémoire[MA]
subx	0x06XX	ACC <- ACC - Mémoire[MA]
ld ADR	0x07XX	ACC <- Mémoire[ADR]
st ADR	0x08XX	Mémoire[ADR] <- ACC
lda ADR	0x09XX	MA <- Mémoire[ADR]
sta ADR	0x0AXX	Mémoire[ADR] <- MA
ldi	0x0BXX	ACC <- Mémoire[MA]
sti	0x0CXX	Mémoire[MA] <- ACC
br ADR	0x0DXX	PC <- ADR
brz ADR	0x0EXX	ACC = 0 ? PC <- ADR : PC <- PC + 1
brnz ADR	0x0FXX	ACC != 0 ? PC <- ADR : PC <- PC + 1
shl	0x10XX	ACC <- ACC << 1
shr	0x11XX	ACC <- ACC >> 1
lea ADR	0x12XX	MA <- ADR
stop	0x13XX	Arrêt du programme

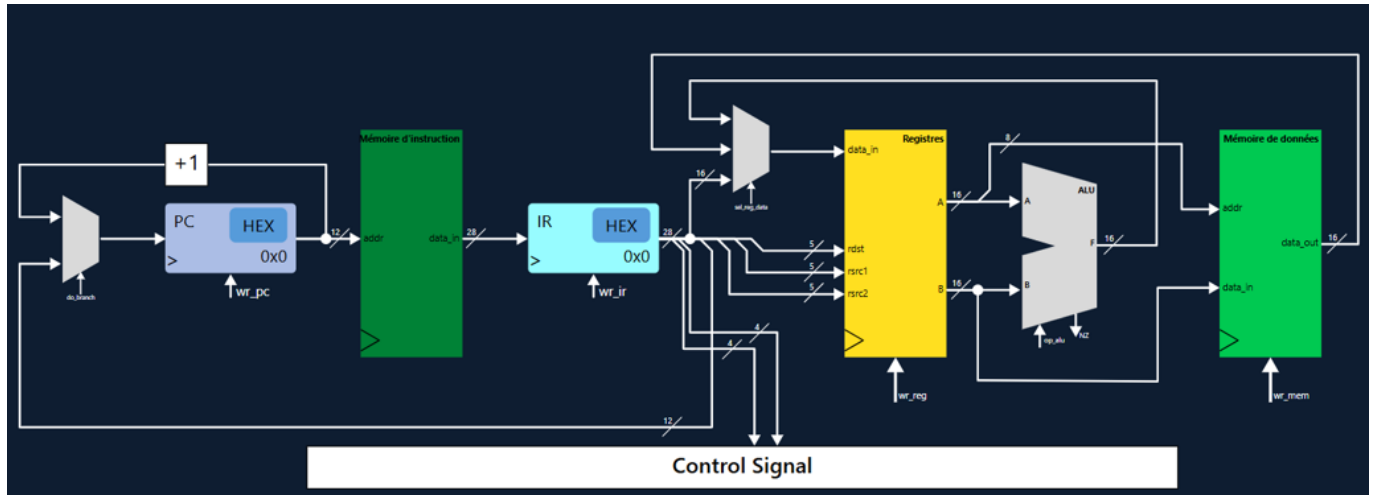
Opérations ALU

op_alu	opération
0	B + A
1	B – A
2	B x A
3	B << 1
4	B >> 1

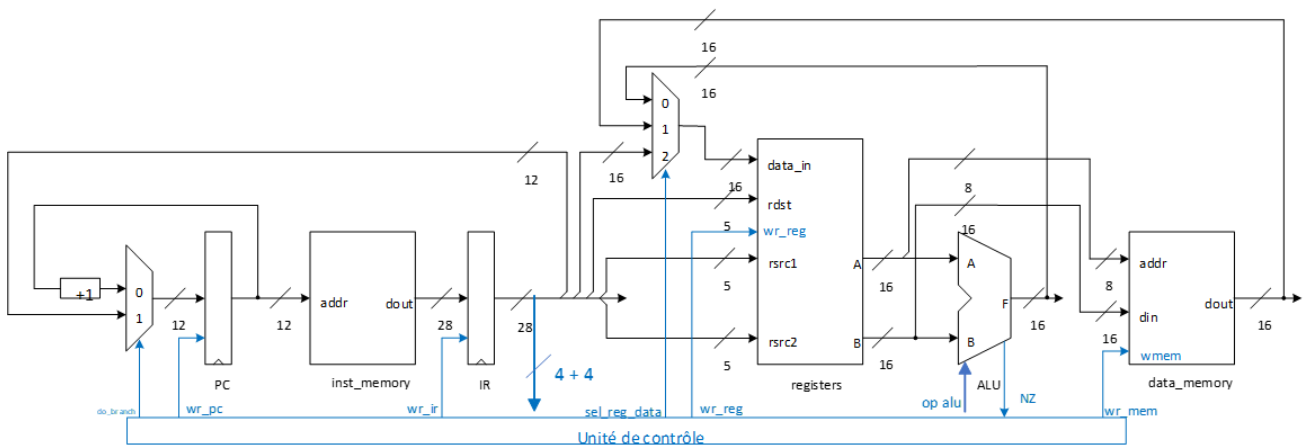
PolyRisc

Circuit logique

CodeMachine



Détaillé



Instructions

alu	0	op_alu		rdst		rsrc1		rsrc2	
	27	26	24	20	16	12	8	4	0
read_mem		1000		rdst		rsrc1			
	27	24	20	16	12	8	4	0	
write_mem		1001				rsrc1		rsrc2	
	27	24	20	16	12	8	4	0	
ldi		1010		rdst				imm	
	27	24	20	16	15				0
jump		1100		jtype				disp	
	27	24	19	16	11				0
stop		1111							
	27	24							

Format des types d'instruction

Types et syntaxe des instructions de l'UAL et de branchements

Champ	Valeur	Type d'opération	Syntaxe assembleur
op	0	Addition arithmétique	add rdst, rsrc1, rsrc2
op	1	Soustraction arithmétique	sub rdst, rsrc1, rsrc2
op	2	Décalage binaire à droite	shr rdst, rsrc1
op	3	Décalage binaire à gauche	shl rdst, rsrc1
op	4	NON logique	not rdst, rsrc1
op	5	ET logique	and rdst, rsrc1, rsrc2
op	6	OU logique	or rdst, rsrc1, rsrc2
op	7	Affectation	mv rdst, rsrc1
jtype	0	Jump non conditionnel	br label
jtype	1	Jump si nul (Z=1)	brz label
jtype	2	Jump si non nul (Z=0)	brnz label
jtype	3	Jump si négatif (N=1)	brlz label
jtype	4	Jump si non négatif (N=0)	brgez label

Types et syntaxe des instructions de lecture, écriture, chargement et d'arrêt

Type d'opération	Syntaxe assembleur
Lecture de la mémoire	ld rdst, (rsrc1)
Ecriture dans la mémoire	st (rsrc1), rsrc2
Chargement d'un immédiat	ldi rdst, imm
Arrêter l'exécution	Stop